SISTEMA DE APROBACIÓN DE PRÉSTAMOS BANCARIOS

Patrón de Diseño: *Chain of Responsibility*

Desacoplamiento de solicitudes mediante una cadena de eslabones jerárquicos y delegación sucesiva

Autor: **Angie Nathaly Cisneros Guerrero**

20242020084

Fecha: Septiembre 2026

Índice

1. Diagrama UML **2**

2. Explicación de las Clases **3**
2.1. Clase de Datos (Prestamo.java) 3
2.2. Clase Abstracta Manejadora (Handler.java) 3
2.3. Manejadores Concretos (EjecutivoCuenta.java, LiderInmediato.java, etc.) . 5

3. Código de Cliente **6**
3.1. Clase Cliente interactiva (Cliente.java) 6

4. Pruebas y Ejecución **9**
4.1. Prueba de Ejecución en Consola 9

5. Repositorio **12**
5.1. Enlace GitHub 12

Diagrama UML

Descripción General

El proyecto implementa una arquitectura basada en el patrón de comportamiento **Chain of Responsibility** (Cadena de Responsabilidad). El sistema permite procesar solicitudes de préstamos bancarios de forma escalonada. La petición es entregada a un primer eslabón y, si este no cuenta con rango monetario para resolverla, la delega automáticamente a su sucesor en la jerarquía (`Ejecutivo` → `LiderEjecutivo` → `Gerente` → `Director`), evitando que el cliente conozca quién maneja finalmente la solicitud.

Diagrama de Clases

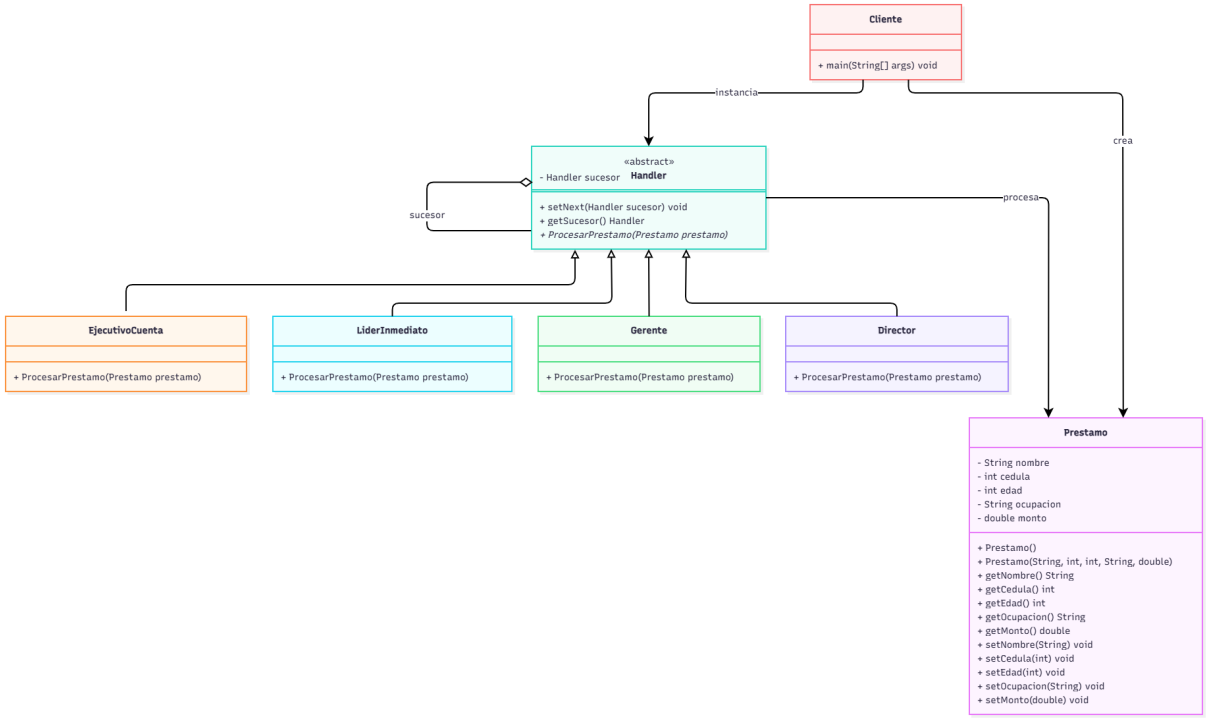


Figura 1: Diagrama de Clases del Sistema (Patrón Chain of Responsibility).

Explicación de las Clases

Clase de Datos (Prestamo.java)

Descripción

Representa el objeto de solicitud que viaja a través de la cadena. Contiene la información personal del solicitante y, de manera crítica, el atributo `monto` que determina qué nivel de autoridad dentro de la organización bancaria debe aprobar la transacción.

Código Fuente: Prestamo.java

```
package banco.chainresponsability;

public class Prestamo {
    private String nombre;
    private int cedula;
    private int edad;
    private String ocupacion;
    private double monto;

    public Prestamo() {}

    public Prestamo(String nombre, int cedula, int edad, String
       ocupacion, double monto) {
        this.nombre = nombre;
        this.cedula = cedula;
        this.edad = edad;
        this.ocupacion = ocupacion;
        this.monto = monto;
    }

    public String getNombre() { return nombre; }
    public int getCedula() { return cedula; }
    public int getEdad() { return edad; }
    public String getOcupacion() { return ocupacion; }
    public double getMonto() { return monto; }

    public void setNombre(String nombre) { this.nombre = nombre;
       }
    public void setCedula(int cedula) { this.cedula = cedula; }
    public void setEdad(int edad) { this.edad = edad; }
    public void setOcupacion(String ocupacion) { this.ocupacion =
        ocupacion; }
    public void setMonto(double monto) { this.monto = monto; }
}
```

Clase Abstracta Manejadora (Handler.java)

Descripción

Define la estructura base para todos los eslabones de la cadena. Contiene una referencia privada/protegida hacia el siguiente elemento de tipo `Handler` (`sucesor`), así como los métodos de enlace (`setSucesort`) y el método abstracto (`ProcesarPrestamo`) que las clases concretas deben implementar obligatoriamente.

Código Fuente: Handler.java

```
/**
 *
 * @author natha
 */
public abstract class Handler {

    protected Handler sucesor;

    //Setter
    public void setSucesor(Handler sucesor) {
        this.sucesor= sucesor;
    }

    //Getter
    public Handler getSucesor() {
        return sucesor;
    }

    public abstract void ProcesarPrestamo(Prestamo prestamo);

}
}
```

Manejadores Concretos (EjecutivoCuenta.java, LiderInmediato.java, etc.)

Descripción

Implementan la lógica de negocio específica para cada rango de aprobación. Si el monto del préstamo entra en su límite de competencia, lo aprueban directamente; de lo contrario, evalúan si existe un sucesor y delegan la solicitud. A continuación se muestran el `Ejecutivo` y el `Director`.

Código Fuente: EjecutivoCuenta.java

```
/**
 *
 * @author natha
 */
public class Ejecutivo extends Handler{

    @Override
    public void ProcesarPrestamo(Prestamo prestamo) {
        double monto = prestamo.getMonto();

        if (monto<= 10000){
            System.out.println("\nPrestamo de $" + monto + " aprobado por el Ejecutivo");
        } else if(sucesor != null){
            System.out.println("Ejecutivo  no puede procesar el prestamo ...");
            getSucesor().ProcesarPrestamo(prestamo);
        }
    }

}
```

Código Fuente: Director.java

```
/**
 *
 * @author natha
 */
public class Director extends Handler{

    @Override
    public void ProcesarPrestamo(Prestamo prestamo) {
        double monto = prestamo.getMonto();
        if(monto>100000){
            System.out.println("Prestamo de $" + monto+ " aprobado por el Director");
        }else {

        System.out.println("Error Credito no aprovado.");
        }
    }
}
```

Código de Cliente

Clase Cliente interactiva (Cliente.java)

Descripción

Configura el encadenamiento inicial de los objetos (`ejecutivo` → `lider` → `gerente` → `director`) y provee una interfaz de consola interactiva mediante `Scanner` para ingresar dinámicamente los datos y montos de los préstamos a evaluar.

Código Fuente: Cliente.java

```
/**
 *
 * @author natha
 */


public class Cliente {

    public static void main(String[] args) {

        Handler ejecutivo = new Ejecutivo();
        Handler lider = new LiderEjecutivo();
        Handler gerente = new Gerente();
        Handler director = new Director();


        ejecutivo.setSucesor(lider);
        lider.setSucesor(gerente);
        gerente.setSucesor(director);

        try (Scanner sc = new Scanner(System.in)) {
            int opcion = 0;

            do {
                System.out.println("\n
                   ========================================");
                System.out.println("   BANCO  DE PRESTAMOS      ")
                   ;
                System.out.println("
                   ========================================");
                System.out.println("1. Solicitar un prestamo");
                System.out.println("2. Salir");
                System.out.print("Seleccione una opcion: ");

                if (sc.hasNextInt()) {
                    opcion = sc.nextInt();
                } else {
                    System.out.println("ingrese un numero valido
                       ");
                    sc.next();
                    continue;
```

```
38                  }
39
40                  if (opcion == 1) {
41                      sc.nextLine();
42
43                      System.out.print("\nIngrese␣el␣nombre␣del␣
                            solicitante:␣");
44                      String nombre = sc.nextLine();
45
46                      System.out.print("Ingrese␣la␣cedula:␣");
47                      int cedula = sc.nextInt();
48
49                      System.out.print("Ingrese␣la␣edad:␣");
50                      int edad = sc.nextInt();
51
52                      sc.nextLine();
53                      System.out.print("Ingrese␣la␣ocupacion:␣");
54                      String ocupacion = sc.nextLine();
55
56                      System.out.print("Ingrese␣el␣monto␣del␣
                            prestamo␣($):␣");
57                      double monto = sc.nextDouble();
58
59
60                      Prestamo prestamo = new Prestamo(nombre,
                            cedula, edad, ocupacion, monto);
61
62                      System.out.println("\n---␣EVALUANDO␣SOLICITUD
                            ␣---");
63
64                      ejecutivo.ProcesarPrestamo(prestamo);
65
66                  } else if (opcion == 2) {
67                      System.out.println("␣Saliendo...");
68                  } else {
69                      System.out.println("Opcion␣no␣valida");
70                  }
71
72              } while (opcion != 2);
73          }
74      }
75  }
```

Flujo de Ejecución

1. El cliente instancia los cuatro niveles jerárquicos y los enlaza mediante el método `setSucesor`.
2. El usuario introduce los datos y el monto del préstamo deseado por consola.
3. Se crea la solicitud y se despacha de forma exclusiva al primer eslabón (`ejecutivo`).

4. La cadena se encarga internamente de escalar la petición hasta hallar el cargo autorizado según las reglas del banco.

Pruebas y Ejecución

Prueba de Ejecución en Consola

Ejemplo de pruebas con distintos montos para verificar la correcta escalabilidad en la cadena de responsabilidad:

Prueba Cliente

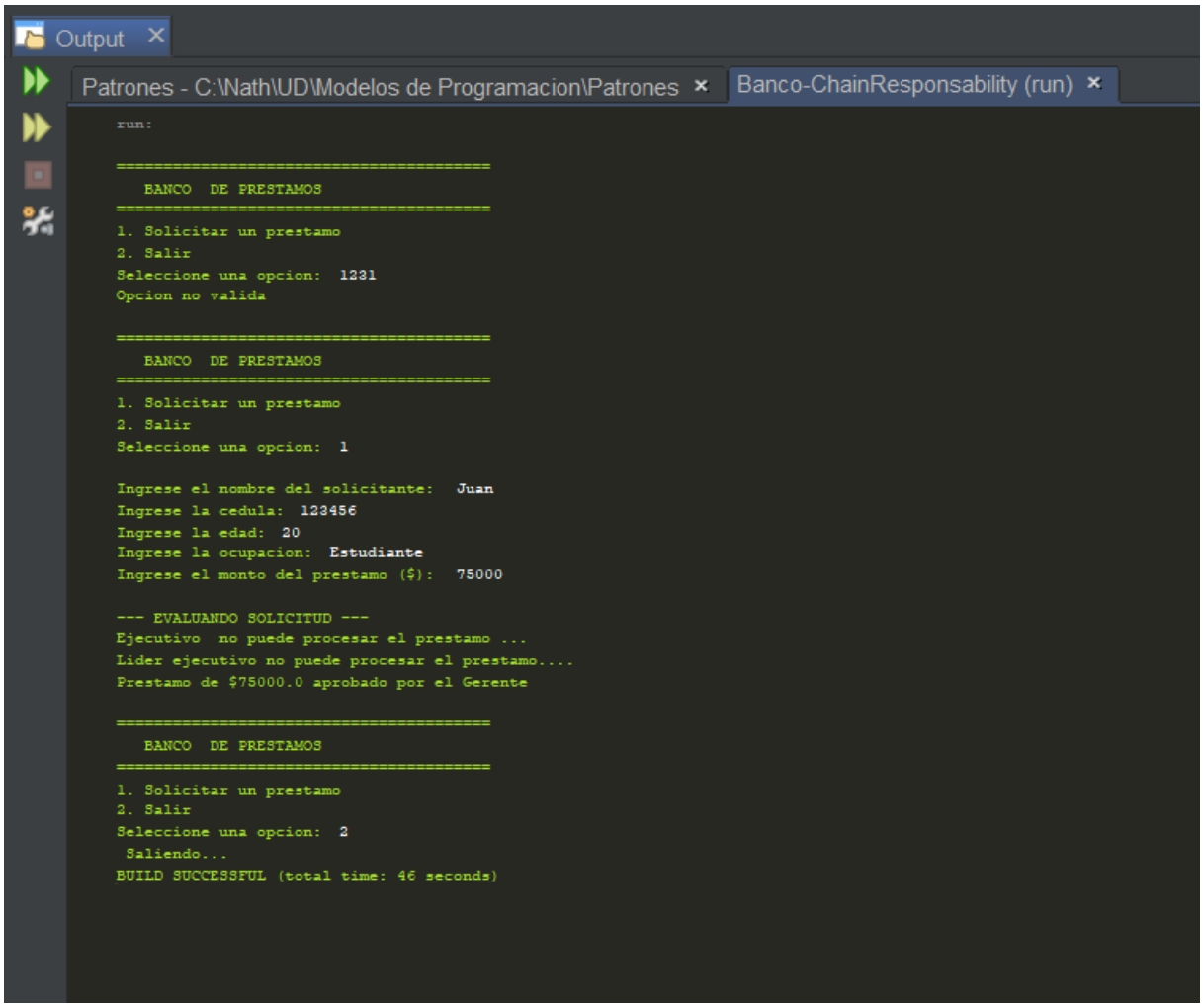


Figura 2: Salida en consola.

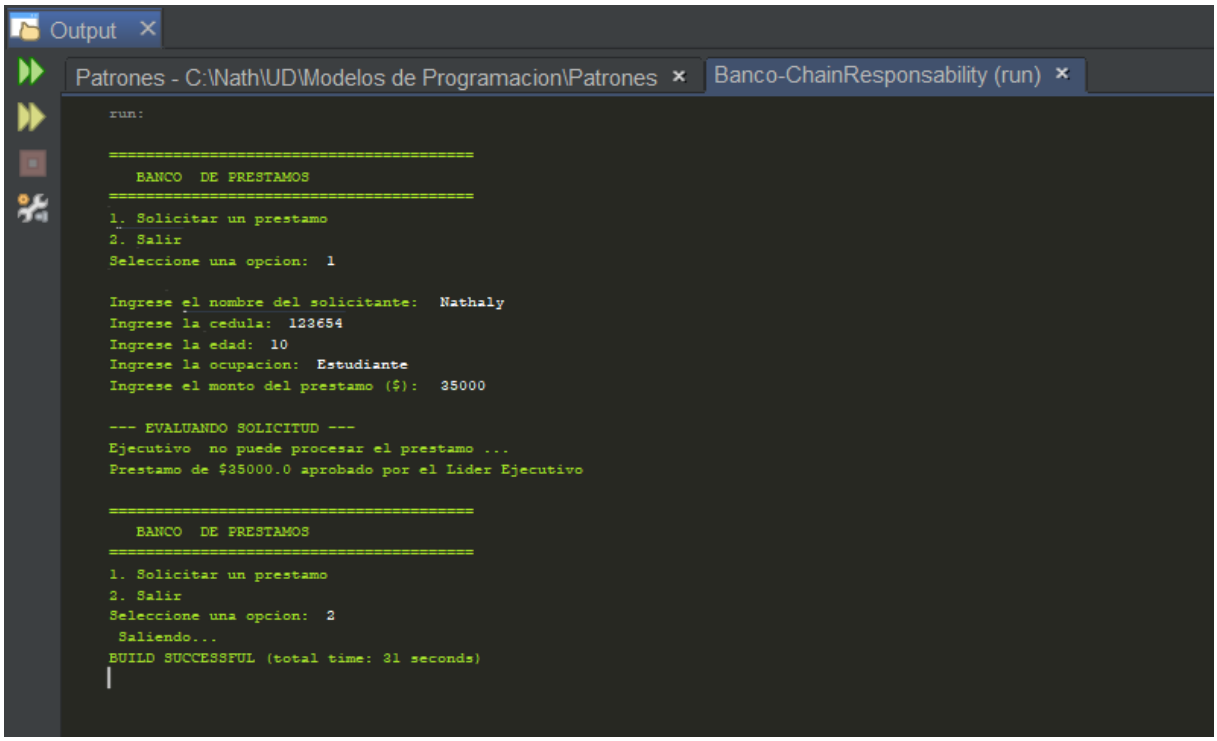


Figura 3: Salida en consola.

Tambien se tiene una prueba unitaria:

```
/**
 *
 * @author natha
 */
public class ClientePrueba {

    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        // TODO code application logic here

        Handler ejecutivo = new Ejecutivo();
        Handler lider = new LiderEjecutivo();
        Handler gerente = new Gerente();
        Handler director = new Director();

        ejecutivo.setSucesor(lider);
        lider.setSucesor(gerente);
        gerente.setSucesor(director);

        //Casos de prueba
        Prestamo prestamo1 = new Prestamo("Carlos Perez", 10123,
           30, "Ingeniero", 8000.0);
        Prestamo prestamo2 = new Prestamo("Ana Gomez", 20456, 35,
            "Doctora", 35000.0);
```

```
25          Prestamo prestamo3 = new Prestamo("Luis␣Torres", 30789,
               42, "Empresario", 75000.0);
26          Prestamo prestamo4 = new Prestamo("Sofia␣Ruiz", 40321,
               50, "Inversionista", 150000.0);
27
28          System.out.println("PROCESANDO␣PRESTAMO␣1:");
29          ejecutivo.ProcesarPrestamo(prestamo1);
30
31          System.out.println("\n␣PROCESANDO␣PRESTAMO␣2:");
32          ejecutivo.ProcesarPrestamo(prestamo2);
33
34          System.out.println("\n␣PROCESANDO␣PRESTAMO␣3␣:");
35          ejecutivo.ProcesarPrestamo(prestamo3);
36
37          System.out.println("\n␣PROCESANDO␣PRESTAMO␣4:");
38          ejecutivo.ProcesarPrestamo(prestamo4);
39      }
40
41  }
```

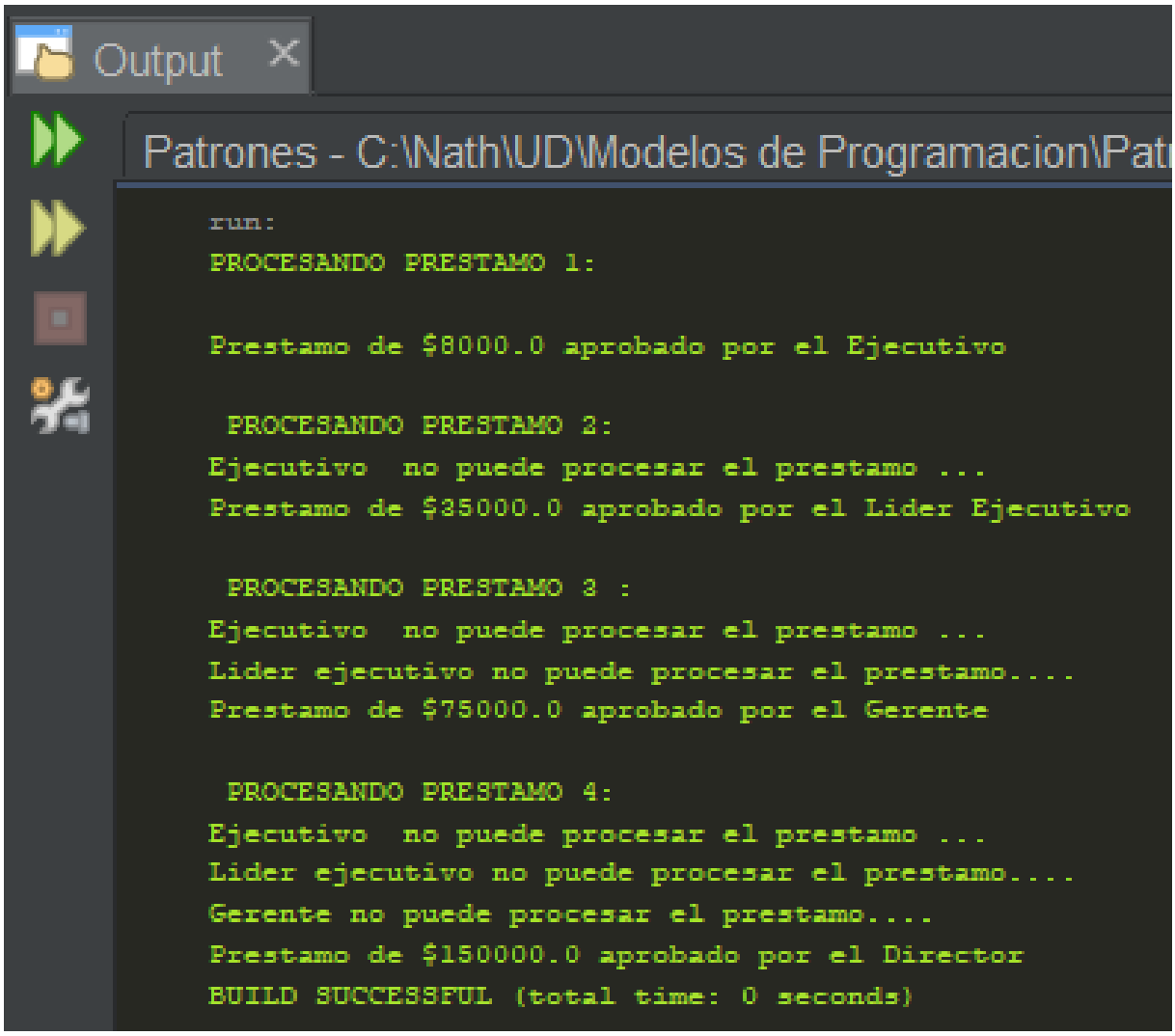


Figura 4: Salida en consola.

Repositorio

Enlace GitHub

- https://github.com/Thalyhx/Modelos-de-Programacion/tree/main/Patrones%20Comportamiento/Banco-ChainResponsability